

Processor caches

(reference: text Sections 5.1-5.4, 5.8)

commonly have **multiple levels** of processor cache

- **primary** (or “**level 1**”) cache, typically split into separate **instruction** and **data** caches (e.g., 32 “KiB” each)
- **secondary** (or “**level 2**”) cache, larger but slower than primary cache (e.g., 2 “MiB”)
- **tertiary** (or “**level 3**”) cache, larger but slower than secondary cache

(typically, single L3 cache *shared* among all cores on same chip, while each core has *own* L1 and L2 caches)

processor cache management

in *general purpose* processors, caches are **managed by the hardware** (“transparently” to application software)

what is put in cache memory?

- **copies** of **main memory words** that are **likely to be needed in near future**

how are cache memory contents accessed?

- on every memory operation, **hardware looks in cache memory first**, to see if there’s a copy there, using as the “**lookup key**” the **main memory address**

locality of reference

how does the **hardware** determine what memory words are “likely to be needed in near future”?

most programs exhibit *locality of reference*:

- “**temporal locality**”: words **recently accessed** are likely to be accessed again in near future
- “**spatial locality**”: words **adjacent in main memory to words recently accessed** are likely to be accessed in near future

what are some common program features that would lead to temporal or spatial locality?

operation for memory reads

hardware looks in primary (level 1) cache first

if “**hit**”, done!

if “**miss**”, look in the next level(s) of cache ... if in none of these, read from main memory

when have the data, **place a copy in the cache(s)** ... likely a good decision, if there is **temporal** locality!

(some details here regarding operation with multiple levels of caches, e.g. “inclusive”, “exclusive”, neither, ...)

blocks

on a miss, hardware retrieves more than just the requested word ... **also retrieve some words adjacent to it in main memory**

likely a good decision, if there is **spatial** locality

- (time to retrieve N words) $\ll$

 - $N \times$ (time to retrieve one word)

 - and so if we do need these other words in near future, better to get them now!

exactly which other words do we get?

statically divide main memory into **fixed-size blocks** (also called **lines**), retrieve **entire block containing requested word**

e.g., for block size of 4 words:

block 0: words with addresses **0, 4, 8, 12**

block 1: words with addresses **16, 20, 24, 28**

block 2: words with addresses **32, 36, 40, 44**

...

if requested **word with address 20**, retrieve **block 1**

operation for memory writes

hardware looks in primary (level 1) cache first

if “**hit**”, two options:

- **write-through** policy: copy in the cache is updated, *and* the write is propagated to the next level
- **write-back** policy: only update copy in cache

with the **write-back** policy, when a “**dirty**” block is *removed* from cache, must “**write back**” the updated contents

if “**miss**”, two options:

- **propagate the write** to the next level
- **write-allocate**: **read entire block** containing word, and then update (in the cached copy) the particular word being written to

when need to write to lower-level cache or to main memory, **don't need to wait for the write to complete!** ... instead, just put in **write buffer** ...